

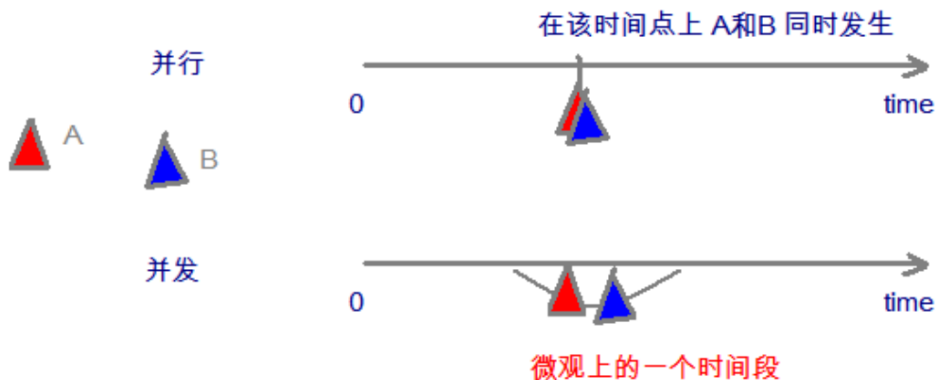
1.多线程入门

我们在之前，学习的程序在没有跳转语句的前提下，都是由上至下依次执行，那现在想要设计一个程序，边打游戏边听歌，怎么设计？

要解决上述问题,咱们得使用多进程或者多线程来解决.

1.1 并发与并行

- **并发**：指两个或多个事件在**同一个时间段内**发生。
- **并行**：指两个或多个事件在**同一时刻**发生（同时发生）。



在操作系统中，安装了多个程序，并发指的是在一段时间内宏观上有多个程序同时运行，这在单 CPU 系统中，每一时刻只能有一道程序执行，即微观上这些程序是分时的交替运行，只不过是给人的感觉是同时运行，那是因为分时交替运行的时间是非常短的。

而在多个 CPU 系统中，则这些可以并发执行的程序便可以分配到多个处理器上（CPU），实现多任务并行执行，即利用每个处理器来处理一个可以并发执行的程序，这样多个程序便可以同时执行。目前电脑市场上说的多核 CPU，便是多核处理器，核 越多，并行处理的程序越多，能大大的提高电脑运行的效率。

注意：单核处理器的计算机肯定是不能并行的处理多个任务的，只能是多个任务在单个CPU上并发运行。同理,线程也是一样的，从宏观角度上理解线程是并行运行的，但是从微观角度上分析却是串行运行的，即一个线程一个线程的去运行，当系统只有一个CPU时，线程会以某种顺序执行多个线程，我们把这种情况称之为线程调度。

1.2 线程与进程

- **进程**：是指一个内存中运行的应用程序，每个进程都有一个独立的内存空间和系统资源，一个应用程序可以同时运行多个进程；进程也是程序的一次执行过程，是系统运行程序的基本单位；系统运行一个程序即是一个进程从创建、运行到消亡的过程。进程是系统进行资源分配和调度的独立单位。
- 单cpu同一时间点只能执行一件事情，CPU高效的切换让我们觉得是同时进行的

我们在同一个进程内可以执行多个任务，每个任务就可以看成一个线程

- **线程**：线程是进程中的一个执行单元，负责当前进程中程序的执行，一个进程中至少有一个线程。一个进程中是可以有多个线程的，这个应用程序也可以称之为多线程程序。

简而言之：一个程序运行后至少有一个进程，一个进程中可以包含多个线程

我们可以再电脑底部任务栏，右键----->打开任务管理器,可以查看当前任务的进程：

单线程:如果程序只有一条执行路径

多线程:如果程序有多条执行路径

多线程存在的意义？

多线程的存在，不是为了提高程序的执行速率，其实是为了提高应用程序的使用率。

程序执行其实是抢夺cpu的资源，cpu执行权

多个进程在抢夺这个资源，而其中某一个进程如果执行路径（线程）比较多，就会有更大的概率抢夺到

cpu的执行权。 我们不敢保证哪一个线程在哪一个时刻一定能抢到cpu的执行权，所以线程的执行是有随机性。

进程

名称	9% CPU	46% 内存	2% 磁盘	0% 网络
应用 (6)				
Google Chrome (32 位)	0.6%	47.8 MB	0.1 MB/秒	0 Mbps
Task Manager	1.2%	18.8 MB	0 MB/秒	0 Mbps
WeChat (32 位)	0%	58.8 MB	0 MB/秒	0 Mbps
Windows 资源管理器	1.0%	56.7 MB	0 MB/秒	0 Mbps
百度浏览器 (32 位)	0.7%	24.7 MB	0.1 MB/秒	0.1 Mbps
有道云笔记 (32 位)	0%	44.4 MB	0 MB/秒	0 Mbps
后台进程 (57)				
Application Frame Host	0%	3.7 MB	0 MB/秒	0 Mbps
bbnet service	0%	1.5 MB	0 MB/秒	0 Mbps
COM Surrogate	0%	1.5 MB	0 MB/秒	0 Mbps
Cortana (小娜)	0%	0.6 MB	0 MB/秒	0 Mbps
Elan Service	0%	0.6 MB	0 MB/秒	0 Mbps
ETD Control Center	0%	2.1 MB	0 MB/秒	0 Mbps

线程



线程调度:

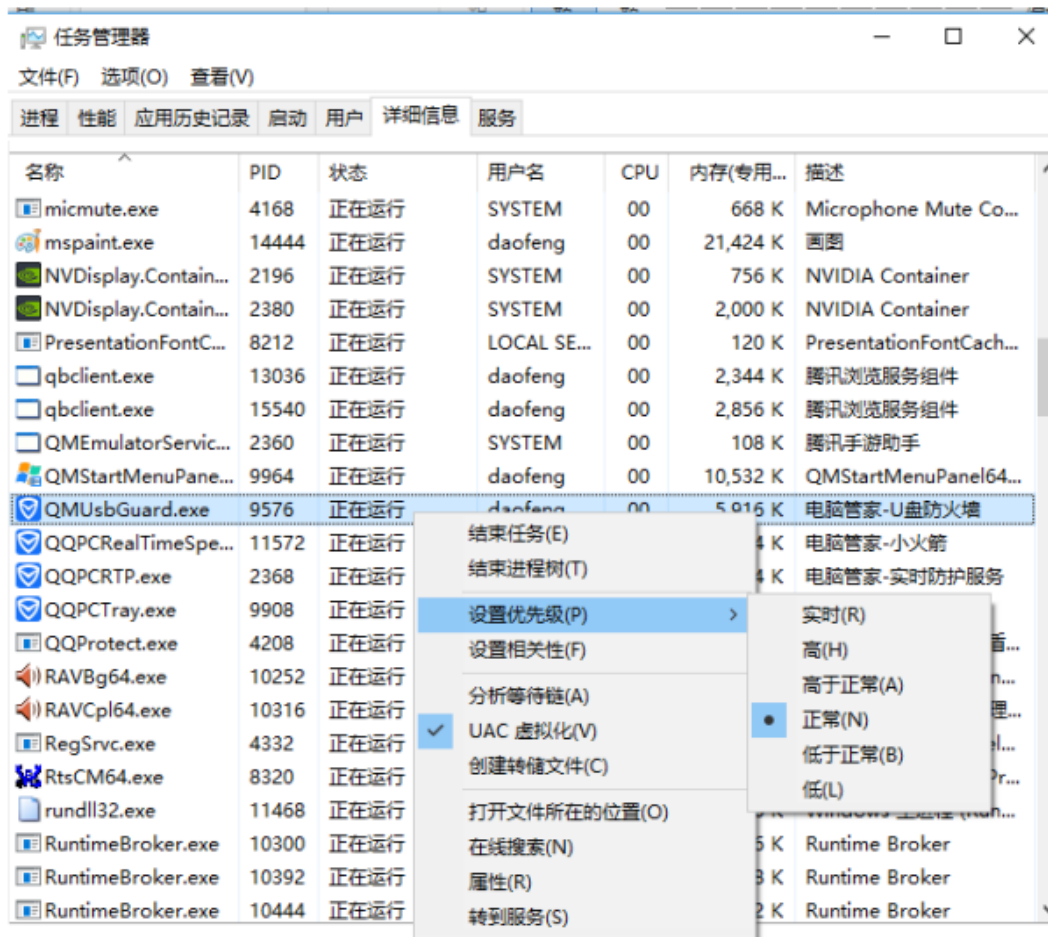
- 分时调度

所有线程轮流使用 CPU 的使用权，平均分配每个线程占用 CPU 的时间。

- 抢占式调度

优先让优先级高的线程使用 CPU，如果线程的优先级相同，那么会随机选择一个(线程随机性)，Java使用的为抢占式调度。

- 设置线程的优先级

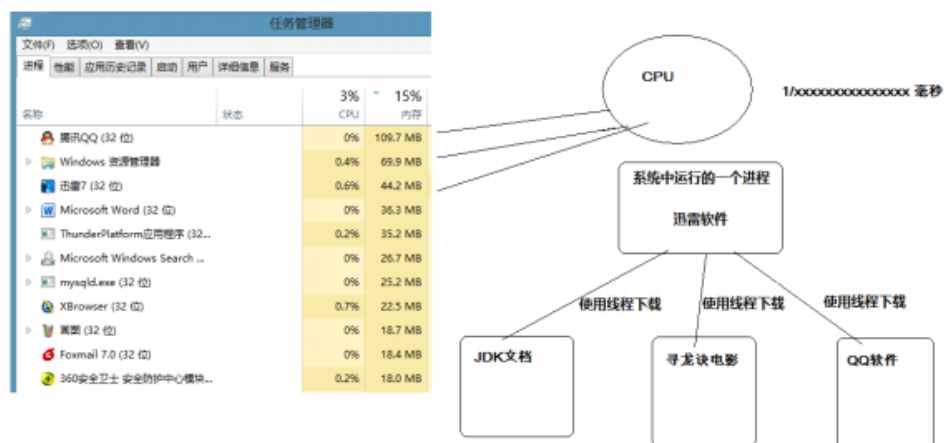


○ 抢占式调度详解

大部分操作系统都支持多进程并发运行，现在的操作系统几乎都支持同时运行多个程序。比如：现在我们上课一边使用编辑器，一边使用录屏软件，同时还开着画图板，dos窗口等软件。此时，这些程序是在同时运行，“感觉这些软件好像在同一时刻运行着”。

实际上，CPU(中央处理器)使用抢占式调度模式在多个线程间进行着高速的切换。对于CPU的一个核而言，某个时刻，只能执行一个线程，而CPU的在多个线程间切换速度相对我们的感觉要快，看上去就是在同一时刻运行。

其实，多线程程序并不能提高程序的运行速度，但能够提高程序运行效率，让CPU的使用率更高。



1. 要想知道多线程，必须先了解线程，要想知道线程，必须先了解进程，因为线程依赖于进程存在的。

2. 什么进程？ 只有运行的程序才会出现进程 进程就是正在运行的程序
进程是系统进行资源分配和调度的独立单位，每一个进程都有它自己的内存空间和系统资源。
多进程有什么意义？ 单进程计算机只能做一件事情

3. 什么是线程？

在同一个进程内可以执行多个任务，而这个每个任务就是看成线程
线程，是程序执行的单元，执行路径，是程序使用cpu最基本单位。

单线程 : 如果程序只有只有一条执行路径 多线程: 如果程序有多条执行路径

多线程意义？

多线程存在的意义: 不是提高程序的执行速率, 其实是为了提高程序的使用率。

程序的执行其实都是在抢cpu的资源，cpu的执行权。

多个线程抢夺到cpu执行权的概述更大 线程抢夺执行权具有随机性

并发 逻辑上同时发生，指再某一个时间内同时运行的程序

并行 物理上同时发生，指在某一个时间点同时运行的程序

java程序的运行原理 由java命令启动jvm, 启动jvm相当于启动了一个进程

接着该进程创建主线程去调用main方法

jvm虚拟机的启动是单线程的还是多线程的？多线程的

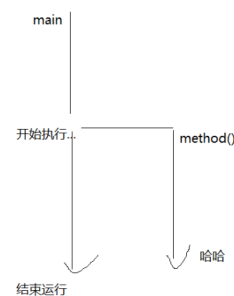
原因是垃圾回收线程也要启动，不然很容易就内存溢出

```
public class Demo01 {  
    public static void main(String[] args) {  
  
        System.out.println("开始执行....");  
        method();  
  
        System.out.println("结束运行....");  
    }  
  
    public static void method() {  
        System.out.println("哈哈....");  
    }  
}
```

单线程 程序只有一条执行路径



多线程程序 多条执行路径



1.3 创建线程类

Java使用 `java.lang.Thread` 类代表**线程**，所有的线程对象都必须是Thread类或其子类的实例。每个线程的作用是完成一定的任务，实际上就是执行一段程序流即一段顺序执行的代码。Java使用线程执行体来代表这段程序流。Java中通过继承Thread类来**创建并启动多线程**的步骤如下：

1. 定义Thread类的子类，并重写该类的run()方法，该run()方法的方法体就代表了线程需要完成的任务,因此把run()方法称为线程执行体。
2. 创建Thread子类的实例，即创建了线程对象
3. 调用线程对象的start()方法来启动该线程

java.lang

类 Thread

[java.lang.Object](#)

└ [java.lang.Thread](#)

所有已实现的接口:

[Runnable](#)

```
public class Thread
extends Object
implements Runnable
```

线程 是程序中的执行线程。Java 虚拟机允许应用程序并发地运行多个执行线程。

```
//1.编写一个类继承Thread
//2.重写run方法，封装多线程要执行的代码
public class MyThread extends Thread {

    //编写多线程执行的代码
    @Override
    public void run() {

        for(int i=0;i<9;i++) {
            System.out.println("奥利给:"+i);
        }

    }

}
```

```
public class MyThreadDemo {

    public static void main(String[] args) {

        //1.编写一个类继承Thread
        //2.重写run方法
        //3.创建线程对象
        //4.调用start方法
        MyThread myThread=new MyThread();

        //调用start方法执行线程
        myThread.start();//使该线程开始执行；Java 虚拟机调用该线程的 run 方法。

        for(int i=0;i<9;i++) {
            System.out.println("给力奥:"+i);
        }

    }

}
```

```
}
```

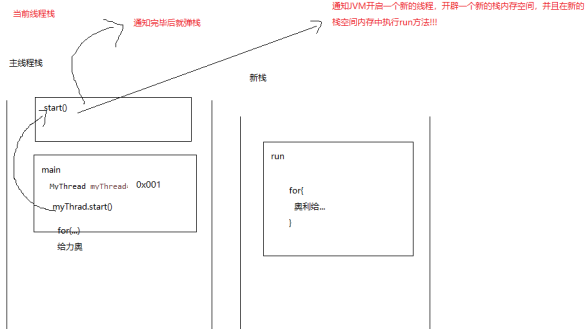
```
}
```

1.4 多线程独立栈空间

```
for(int i=0;i<9;i++) {  
    System.out.println("给力奥:"+i);  
}
```

```
public class MyThreadDemo {  
    public static void main(String[] args) {  
        MyThread myThread=new MyThread();  
        myThread.start();  
        for(int i=0;i<9;i++) {  
            System.out.println("给力奥:"+i);  
        }  
    }  
}
```

每个线程都有自己独立的栈空间



1.5 多线程的打印具备随机性

JVM执行main方法，操作系统开辟一条通向cpu的路径，这个路径叫做main线程，主线程

cpu通过这个主线程来执行main方法

start() 通知jvm开启一个新的线程，来执行run方法

cpu 虚拟机调用该线程的 run 方法。

执行run方法

开辟了一条通往cpu的新道路来执行run方法

```
public class MyThreadDemo {  
    public static void main(String[] args) {  
        MyThread myThread=new MyThread();  
        //调用start方法执行线程  
        myThread.start();  
        for(int i=0;i<9;i++) {  
            System.out.println("给力奥:"+i);  
        }  
    }  
}
```

```
public class MyThread extends Thread {  
    //编写多线程执行的代码  
    @Override  
    public void run() {  
        for(int i=0;i<9;i++) {  
            System.out.println("奥利给:"+i);  
        }  
    }  
}
```

给力奥:0
奥利给:0
给力奥:1
奥利给:1
给力奥:2
奥利给:2
给力奥:3
给力奥:4
给力奥:5
给力奥:6
给力奥:7
给力奥:8
奥利给:3

对于cpu而言，天哪，我到底执行谁？
就有两条路径，cpu具有选择权。
cpu喜欢谁，就会执行哪条路径，我们控制不了cpu
所以就有了随机打印的结果

两个线程，一个main线程，一个子线程抢夺cpu的执行权（执行时间），
谁抢到了就执行谁。

1.6 创建多个子线程

```
public class MyThreadDemo {  
  
    public static void main(String[] args) {
```

//创建多个线程不是多次调用start方法
//而是创建多个Thread对象，然后分别开启线程

```
MyThread myThread=new MyThread();
```

```

//调用start方法执行线程
myThread.start();//使该线程开始执行；Java 虚拟机调用该线程的 run 方法。

MyThread myThread2=new MyThread();
myThread2.start();

for(int i=0;i<9;i++) {
    System.out.println("给力奥:"+i);
}

}

}

```

1.7 获得线程的名字

可以在Thread的对象中直接使用

```
String getName()
返回该线程的名称。
```

如果在其他类中如何获取当前线程的名字

currentThread

public static Thread currentThread()返回对当前正在执行的线程对象的引用。

```

//获取当前线程对象
Thread currentThread = Thread.currentThread();
currentThread.getName();//获取名字

```

1.8 线程优先级(了解)


```

/**
 * The minimum priority that a thread can have.
 */
public static final int MIN_PRIORITY = 1; 最小

/**
 * The default priority that is assigned to a thread.
 */
public static final int NORM_PRIORITY = 5; 默认

/**
 * The maximum priority that a thread can have.
 */
public static final int MAX_PRIORITY = 10; 最大

...

```

```

public class ThreadPriority extends Thread{

    //Thread(String name)
    //分配新的 Thread 对象
    public ThreadPriority(String name) {
        super(name);
    }

    @Override
    public void run() {

        for(int i=0;i<100;i++) {
            System.out.println(getName()+"我是奥利给!");
        }

    }

}

```

```

public class ThreadPriorityDemo {

    public static void main(String[] args) {

        ThreadPriority t1=new ThreadPriority("小龙");

        ThreadPriority t2=new ThreadPriority("二龙");

        ThreadPriority t3=new ThreadPriority("子龙");

        //设置优先级
    }

}

```

```

        //public final void setPriority(int newPriority)
        //让二龙先跑
        t2.setPriority(9);

        //int getPriority()
        //返回线程的优先级。
        //默认的优先级都是5
        System.out.println(t1.getPriority());
        System.out.println(t2.getPriority());
        System.out.println(t3.getPriority());

        t1.start();
        t2.start();
        t3.start();

    }

}

```

2.线程控制

2.1 线程的休眠 ***

```
static void sleep(long millis)    millis毫秒 1秒=1000ms
```

在指定的毫秒数内让当前正在执行的线程休眠（暂停执行），此操作受到系统计时器和调度程序精度和准确性的影响。

```

public class ThreadSleep extends Thread{

    public ThreadSleep(String name) {
        super(name);
    }

    @Override
    public void run() {

        for(int i=0;i<9;i++) {

            System.out.println(getName()+"我是奥利给!");

            //我很困，我要休息3秒在执行下一次循环
            //static void sleep(long millis)
            try {
                Thread.sleep(3000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

```

```

        }

    }

}

}

```

```

public class ThreadSleepDemo {

    public static void main(String[] args) {

        ThreadSleep t1=new ThreadSleep("小龙1");
        ThreadSleep t2=new ThreadSleep("小龙2");
        ThreadSleep t3=new ThreadSleep("小龙3");

        t1.start();
        t2.start();
        t3.start();

    }

}

```

2.2 线程加入

```

public class ThreadJoin extends Thread{

    public ThreadJoin(String name) {
        super(name);
    }

    @Override
    public void run() {

        for(int i=0;i<9;i++) {

            System.out.println(getName()+"我是奥利给!");

```

```
    }  
  
    }  
  
}
```

```
public class ThreadJoinDemo {  
  
    public static void main(String[] args) {  
  
        ThreadJoin t1=new ThreadJoin("小龙");  
        ThreadJoin t2=new ThreadJoin("龙女");  
        ThreadJoin t3=new ThreadJoin("二龙");  
  
        t1.start();  
  
        //    void join()  
        //    等待该线程终止。  
  
        //    等待该线程先执行多少时间  
        //public final void join(long millis)  
        try {  
            //等待该线程执行完毕其它线程才能接着运行  
            t1.join();  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
  
        t2.start();  
        t3.start();  
  
    }  
  
}
```

2.3 线程的礼让

```
public static void yield()
```

暂停当前正在执行的线程对象，并执行其他线程。

```
public class ThreadYield extends Thread{

    public ThreadYield(String name) {
        super(name);
    }

    @Override
    public void run() {

        for(int i=0;i<9;i++) {

            System.out.println(getName()+"我是奥利给!");

            //暂停当前正在执行的线程对象，并执行其他线程。
            Thread.yield();

        }

    }

}
```

```
public class ThreadYieldDemo {

    public static void main(String[] args) {

        ThreadYield t1=new ThreadYield("小龙");
        ThreadYield t2=new ThreadYield("龙女");
        ThreadYield t3=new ThreadYield("二龙");

        t1.start();

    }

}
```

```
        t2.start();
        t3.start();

    }

}
```

2.4 守护线程

守护主线程的执行，如果主线程执行完毕，守护线程也会终止运行

```
public class ThreadDaemon extends Thread{

    public ThreadDaemon(String name) {
        super(name);
    }

    @Override
    public void run() {

        for(int i=0;i<100;i++) {

            System.out.println(getName()+"我是奥利给!" + i);

            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }

        }

    }

}
```

```

public class ThreadDaemonDemo {

    public static void main(String[] args) {

        //如何修改主线程的名字?
        Thread.currentThread().setName("小龙女朋友");

        ThreadDaemon t1=new ThreadDaemon("小龙");
        ThreadDaemon t2=new ThreadDaemon("岳父");

        //void setDaemon(boolean on)
        //将该线程标记为守护线程或用户线程。

        //守护主线程的执行，如果主线程执行完毕，守护线程也会终止运行
        t1.setDaemon(true);
        t2.setDaemon(true);

        t1.start();

        t2.start();

        for(int i=0;i<9;i++) {

            System.out.println(Thread.currentThread().getName()+"奥利给"+i);

        }

    }

}

```

2.5 线程停止 *

```

public class ThreadStop extends Thread{

    public ThreadStop(String name) {
        super(name);
    }

    @Override
    public void run() {

        try {
            Thread.sleep(9000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

    }

}

```

```
        for(int i=0;i<100;i++) {

            System.out.println(getName()+"我是奥利给!" +i);

        }

    }

}
```

```
public class ThreadDaemonDemo {

    public static void main(String[] args) {

        ThreadStop t1=new ThreadStop("小龙");

        t1.start();

        try {

            //主线我等你等3秒
            Thread.sleep(3000);

            //线程停止
            // void stop() 停止一个线程
            // 已过时。
            t1.stop();

        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println("主线程运行结束...");

    }

}
```



```
}
```

```
public class ThreadStop extends Thread{

    public ThreadStop(String name) {
        super(name);
    }

    @Override
    public void run() {

        try {
            Thread.sleep(9000);
        } catch (InterruptedException e) {
            e.printStackTrace();
            //中断后的代码可以自己定义
            return;
        }

        for(int i=0;i<100;i++) {

            System.out.println(getName()+"我是奥利给!" + i);

        }

    }

}
```

```
public class ThreadDaemonDemo {

    public static void main(String[] args) {

        ThreadStop t1=new ThreadStop("小龙");
```

```

t1.start();

try {

    //主线我等你等3秒
    Thread.sleep(3000);

    //线程停止
    // void stop() 停止一个线程
    // 已过时。
    //t1.stop();

    //public void interrupt() 中断睡眠
    t1.interrupt();

} catch (InterruptedException e) {
    e.printStackTrace();
}

System.out.println("主线程运行结束...");

}

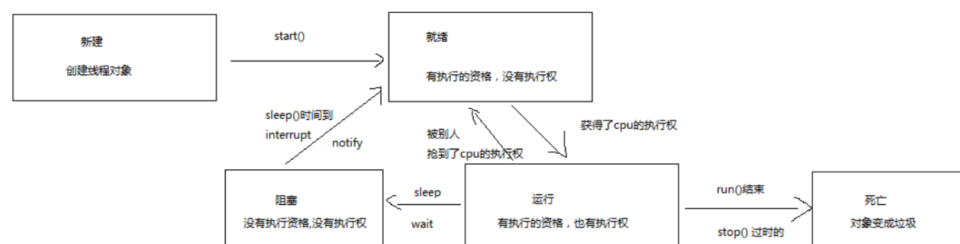
}

```

2.6 多线程的生命周期图

新建: 创建线程对象
就绪: 有执行的资格, 没有执行权
运行: 有执行的资格, 有执行权
阻塞: 由于一些操作让线程处于该状态。没有执行的资格, 没有执行权
而另外一些操作却可以把它给激活, 激活过后处于就绪状态
死亡: 线程对象变成了垃圾, 等待回收

多线程的生命周期图



3. Runnable创建多线程

3.1 runnable方式

```

//1. 编写一个类实现Runnable接口
//2. 实现run方法

```

```
//这种方式的本质是我们把需要多线程执行的代码封装到Runnable对象run方法中，
//然后把run方法通过Runnable对象往Thread的构造器中进行传递，覆盖Thread自己的run方法
public class MyRunnable implements Runnable{

    @Override
    public void run() {

        for(int i=0;i<9;i++) {

            System.out.println(Thread.currentThread().getName()+"奥利给好"+i);

        }

    }

}
```

```
public class MyRunnableDemo {

    public static void main(String[] args) {

        //1.编写一个类实现Runnable接口
        //2.实现run方法
        //3.创建Runnable对象
        //4.创建Thread对象，把Runnable对象作为构造函数的参数传递给了Thread对象
        //5.调用Thread对象的start方法开启线程

        //1.创建Runnable对象
        MyRunnable runnable=new MyRunnable();

        //2.Thread(Runnable target)
        //分配新的 Thread 对象。

        Thread t1=new Thread(runnable);
        t1.start();

        Thread t2=new Thread(runnable);
        t2.start();

    }

}
```

3.2 创建多线程的方式

面试题： 创建多线程的方式有哪些？

方式1: 继承Thread

1. 编写一个类继承Thread
2. 重写run方法
3. 创建线程对象
4. 调用start方法

方式2: 实现Runnable接口

1. 编写一个类实现Runnable接口
2. 实现run方法
3. 创建Runnable对象
4. 创建Thread对象, 把Runnable对象作为构造函数的参数传递给了Thread对象
5. 调用Thread对象的start方法开启线程

为什么有了第一种还会有第二种方式?

A: 可以避免Java单继承的带来的局限性

B: 适合多个相同类型的线程处理同一个资源的情况, 把线程运行的代码和线程本身进行了分离

面试题: 开启一个线程是调用run方法还是start方法?

run方法是封装我们多线程要执行的代码, 如果直接调用就是普通的调用方法

start方法是通知jvm开启一个新的线程来执行run方法

4.线程安全问题

4.1 卖票Thread

```
//大片 你的婚礼
public class SellTicket extends Thread{

    //共享资源:多个线程公共操作一个一个变量

    //100张票 成员变量 属于对象
    //private int tickets=100;

    //静态修饰 属于类的 多个线程共享的一个变量
    private static int tickets=100;

    @Override
    public void run() {

        while(true) {

            if(tickets>0) {

                System.out.println(getName()+"正在出售第"+tickets+"票");
```

```

        tickets--;

    }

}

}

}

```

```

public class SellTicketDemo {

    public static void main(String[] args) {

        //创建了三个Thread对象,而票现在是一个成员变量,属于对象的
        //三个成员变量,相互之前没有关系

        //共享资源:多个线程公共操作一个一个变量

        SellTicket t1=new SellTicket();
        SellTicket t2=new SellTicket();
        SellTicket t3=new SellTicket();

        t1.setName("窗口1");
        t2.setName("窗口2");
        t3.setName("窗口3");

        t1.start();
        t2.start();
        t3.start();

    }

}

```

4.2 卖票Runnable

```

public class SellTicketRunnable implements Runnable{

    //成员变量 属于对象

    //方式2 加不加静态关键字都可以的,因为Runnable对象只创建了一个
    private int tickets=100;

```

```

@Override
public void run() {

    while(true) {

        if(tickets>0) {

            System.out.println(Thread.currentThread().getName()+"正在出售
第"+tickets+"票");
            tickets--;

        }

    }

}

}

```

```

public class SellTicketDemo {

    public static void main(String[] args) {

        //1.创建Runnable对象
        //Runnable只创建了一个对象 只有一个成员变量
        SellTicketRunnable runnable=new SellTicketRunnable();

        //适合多个线程处理共享资源

        //2.创建Thread对象
        //Thread(Runnable target, String name)
        Thread t1=new Thread(runnable,"窗口1");
        Thread t2=new Thread(runnable,"窗口2");
        Thread t3=new Thread(runnable,"窗口3");

        t1.start();
        t2.start();
        t3.start();

    }

}

```

4.3 出现同票的原因

```

//大片 你的婚礼
public class SellTicket extends Thread{

    //共享资源:多个线程公共操作一个一个变量

    //100张票 成员变量 属于对象
    //private int tickets=100;

    //静态修饰 属于类的 多个线程共享的一个变量
    private static int tickets=100;

    @Override
    public void run() {

        while(true) {

            //分析同票的原因

            //线程抢夺cpu的执行权，执行代码的时候具有一个原子操作
            //原子操作是指一个最基本的代码
            //int i=0;

            //i++; 不是原子性的操作
            //i=i+1;

            //t1 t2 t3 tickets=100

            if(tickets>0) {
                //t1抢夺到了cpu的执行权 t2抢夺到了cpu的执行权 t3抢夺到了cpu的执行权

                //t1抢夺到了cpu的执行权 打印100 t2打印100 t3打印100
                System.out.println(getName()+"正在出售第"+tickets+"票");

                //t1还没有执行--操作，t2抢夺到了cpu的执行权 t2还没有执行--，t3抢夺到了cpu的执行权

                tickets--;

            }

        }

    }

}

```

4.4 出现负票的原因

```
public class SellTicketRunnable implements Runnable{

    //成员变量 属于对象

    //方式2 加不加静态关键字都可以的，因为Runnable对象只创建了一个
    private int tickets=100;

    @Override
    public void run() {

        while(true) {

            //t1 t2 t3

            //假设这一次 tickets=1

            if(tickets>0) { //t1抢到了 t2抢到了,并且成功的通过了if的代码

                //t1继续抢到了
                System.out.println(Thread.currentThread().getName()+"正在出售
第"+tickets+"票");

                //t1还没有减除票的库存，就被t2抢夺到了cpu的执行权

                //t1继续减去库存

                try {
                    Thread.sleep(100);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }

                tickets--;//tickets=0

            }

        }

    }

}
```

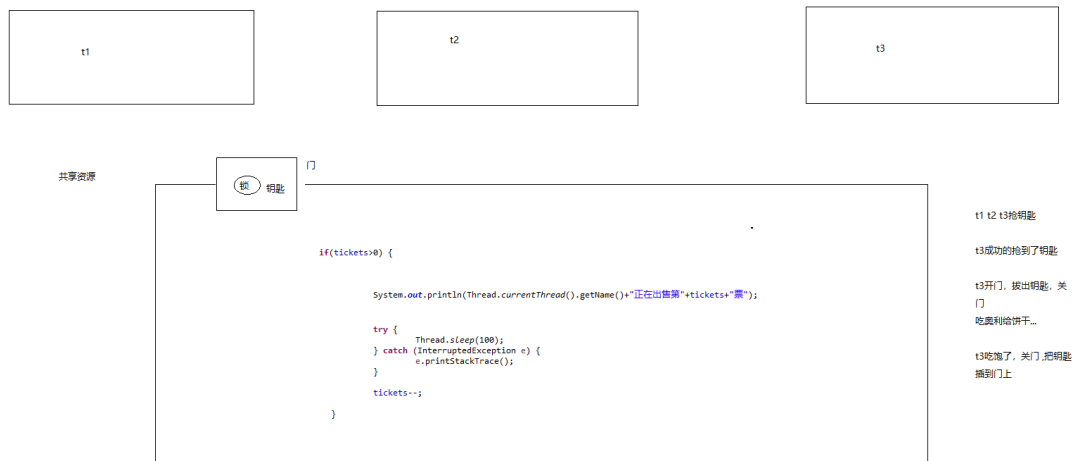

4.5 如何解决线程安全问题(同步代码块)

要想解决线程安全问题？ 就应该考虑哪些情况会导致线程安全问题？

A: 是否多线程的环境

B: 是否有共享资源

C: 是否有多条语句操作共享资源



runnable方式

```
public class SellTicketRunnable implements Runnable{  
  
    //成员变量 属于对象  
  
    //方式2 加不加静态关键字都可以的, 因为Runnable对象只创建了一个  
    private int tickets=100;  
  
    //runnable只创建一个, 那么锁对象只有一个  
    //private Object obj=new Object();  
  
    @Override  
    public void run() {  
  
        while(true) {  
  
            //同步代码块  
            //锁对象任意给一个对象都可以, 但是这个对象必须唯一  
            //this代表当前对象  
            synchronized (this) { //synchronized(锁){需要同步的代码 }  
  
                //t1 t2 t3
```

```

//假设这一次 tickets=1

if(tickets>0) { //t1抢到了 t2抢到了,并且成功的通过了if的代码

    //t1继续抢到了
    System.out.println(Thread.currentThread().getName()+"正在出售
第"+tickets+"票");

    //t1还没有减除票的库存,就被t2抢夺到了cpu的执行权

    //t1继续减去库存

    try {
        Thread.sleep(100);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }

    tickets--;//tickets=0

}

}

}

}

}
}

```

Thread方式

```

//大片 你的婚礼
public class SellTicket extends Thread{

    //共享资源:多个线程公共操作一个一个变量

    //100张票 成员变量 属于对象
    //private int tickets=100;

    //静态修饰 属于类的 多个线程共享的一个变量
    private static int tickets=100;

    //如果在Thread中不能使用非静态成员变量作为锁对象 因为锁对象会创建多个
    //private Object obj=new Object();

    //private static Object obj=new Object();

    @Override
    public void run() {

```

```

while(true) {

    //在Thread中不能使用this作为锁
    //SellTicket.class获取到当前类的字节码对象
    //一个类只有一个字节码对象
    synchronized (SellTicket.class) {

        //分析同票的原因

        //线程抢夺cpu的执行权，执行代码的时候具有一个原子操作
        //原子操作是指一个最基本的代码
        //int i=0;

        //i++; 不是原子性的操作
        //i=i+1;

        //t1 t2 t3    tickets=100

        if(tickets>0) { //t1抢夺到了cpu的执行权    t2抢夺到了cpu的执行权    t3抢夺到了
cpu的执行权

            //t1抢夺到了cpu的执行权    打印100    t2打印100    t3打印100
            System.out.println(getName()+"正在出售第"+tickets+"票");

            //t1还没有执行--操作，t2抢夺到了cpu的执行权    t2还没有执行--，t3抢夺到了
cpu的执行权

            tickets--;

        }

    }

}

}

```

同步方法

Lock锁

集合

arraylist

HashMap

1.2万 以上

juc并发编程

生产者消费者

死锁

java内存模型 原子性 可见性 重排序

信号量

队列

线程池 自定义线程池

悲观锁 乐观锁 读写锁 排他锁 自旋锁

4.6 同步方法

4.6.1 runnable中使用

```
public class SellTicketRunnable implements Runnable{

    //成员变量 属于对象

    //方式2 加不加静态关键字都可以的，因为Runnable对象只创建了一个
    private int tickets=100;

    @Override
    public void run() {

        while(true) {

            //synchronized (this) { //太麻烦了，我给你提供一个简便的写法

            sellTicket();

            // }
        }
    }
}
```

```

    }

}

//同步方法(简便写法,赠送一把锁) 锁是谁? 非静态的方法是属于对象的, 赠送this
public synchronized void sellTicket() {

    if(tickets>0) {

        System.out.println(Thread.currentThread().getName()+"正在出售
第"+tickets+"票");

        tickets--;

    }
}

}

```

4.6.2 Thread中使用

```

//大片 你的婚礼
public class SellTicket extends Thread{

    private static int tickets=100;

    @Override
    public void run() {

        while(true) {

            //            synchronized (SellTicket.class) {

                sellTicket();

            //        }

        }

    }

    //同步方法(简便写法,赠送一把锁) 锁是谁? 非静态的方法是属于对象的, 赠送this锁
    //静态方法赠送SellTicket.class 赠送当前类的字节码对象锁
    //Thread中同步方法只能用静态方法
    public synchronized static void sellTicket() {

        if(tickets>0) {

```

```

        System.out.println(Thread.currentThread().getName()+"正在出售
第"+tickets+"票");

        tickets--;

    }
}

}

```

4.7 Lock锁

```

public class SellTicketRunnable implements Runnable{

    private int tickets=100;

    //创建锁对象
    private Lock lock=new ReentrantLock();

    @Override
    public void run() {

        while(true) {

            //java.util.concurrent 这个包下类处理高并发    juc编程

            //加锁
            lock.lock();

            if(tickets>0) {

                System.out.println(Thread.currentThread().getName()+"正在出售
第"+tickets+"票");

                tickets--;

            }

            //解锁
            lock.unlock();

```

```
    }  
}  
  
}
```

4.8 synchronized和Lock区别

synchronized jvm级别的锁,jvm自动上锁和解锁

Lock锁java代码写的锁，需要手动的加锁和释放锁